6 Laboratory 6

6. Feature points

6.1 Main objectives:

- introduction to the problem of detecting feature points,
- implementation of a simple detection algorithm - Harris method,
- learning how to describe a feature point,
- the surroundings of a point as a descriptor,
- comparison of the Harris, SIFT and ORB (FAST+BRIEF) methods.

R In today's exercise, please use the functions from `matplotlib.pyplot` instead of OpenCV to display images.

6.2 Harris corner detection – theory

Harris corner detection is based on finding pixels for which the vertical and horizontal gradient modulus has a significant value.
The method is described by the following relation:

$$H(x,y) = \det\big(M(x,y)\big) - k\,\mathrm{trace}^2\big(M(x,y)\big) \tag{6.1}$$

where:

- $\det(\cdot)$ is the matrix determinant,
- $\mathrm{trace}(\cdot)$ is the matrix trace,
- k is an empirical constant (typically about 0.04–0.06),
- (x,y) are the image (pixel) coordinates.

The autocorrelation matrix M is:

$$M(x,y) = \begin{bmatrix} \langle I_x^2(x,y)\rangle & \langle I_{xy}(x,y)\rangle \\ \langle I_{xy}(x,y)\rangle & \langle I_y^2(x,y)\rangle \end{bmatrix} \tag{6.2}$$

where each symbol means:

$$\langle I_x^2(x,y)\rangle = I_x^2(x,y) \otimes h(x,y), \tag{6.3}$$
$$\langle I_y^2(x,y)\rangle = I_y^2(x,y) \otimes h(x,y), \tag{6.4}$$
$$\langle I_{xy}(x,y)\rangle = (I_x I_y)(x,y) \otimes h(x,y) \tag{6.5}$$

where:

- $I_x(x,y)$ and $I_y(x,y)$ are the image partial derivatives in the x and y directions (e.g. computed with the Sobel operator),
- $I_{xy}(x,y) = I_x(x,y)\, I_y(x,y)$,
- $\otimes$ denotes convolution,
- $h(x,y)$ is the Gaussian kernel (a weight mask) used to compute a local neighbourhood average.

$$h(x,y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right) \tag{6.6}$$

where:

- σ is the standard deviation (it controls the Gaussian "width").

Gaussian filtering reduces the impact of noise, which can strongly affect gradient estimation.

The determinant and trace of a matrix 2×2 (as a reminder):

$$\det(M(x,y)) = \langle I_x^2(x,y)\rangle \langle I_y^2(x,y)\rangle - \langle I_{xy}(x,y)\rangle^2, \tag{6.7}$$
$$\operatorname{trace}(M(x,y)) = \langle I_x^2(x,y)\rangle + \langle I_y^2(x,y)\rangle \tag{6.8}$$

Finally, a given pixel is classified as a corner if the calculated value $H(x,y)$ is greater than a certain threshold.

As a final filtering it is useful to use the search for local maxima. We consider as a corner candidate only those pixels that are local maxima in their neighbourhood (e.g. 7×7).

6.3 Implementation of the Harris method

Exercise 6.1 Based on the above theory, implement the Harris method.

1. Download the archive with the exercise data from the course website and extract it into your own working directory.
2. **Load the images** *fontanna1.jpg* and *fontanna2.jpg*.
3. **Implement a function** that computes the value H from the autocorrelation matrix. The function should take as parameters a greyscale image and the size of the Sobel and Gaussian filter masks.
   - Compute the appropriate products of directional derivatives and blur them using a Gaussian filter, in accordance with the formulas given above.
   - Assume that the sizes of the Sobel and Gaussian masks can be the same.
   - Assume the coefficient value `K = 0.05`.
   - Normalise the resulting H image to the range 0–1.
4. **Use** the following function, which finds local maxima in the array passed as a parameter:

```
import scipy.ndimage.filters as filters
```

```
def find_max(image, size, threshold) : # size - maximum filter mask size
    data_max = filters.maximum_filter(image, size)
    maxima = (image == data_max)
    diff = image > threshold
    maxima[diff == 0] = 0
    return np.nonzero(maxima)
```

5. For both loaded images, apply the above functions.
   - You may assume mask sizes of 7 in both functions.
   - Display the results from the second function as symbols (e.g. *) overlaid on the original images.

   R It is advisable to create a separate drawing function.
6. **Display and verify** whether the detected points in both images are located (approximately) in the same positions (i.e. whether the detector is repeatable).
7. **Repeat** the operations from the above points for the images *budynek1.jpg* and *budynek2.jpg*.

■

6.4 Simple descriptor of feature points

Exercise 6.2 This exercise is a continuation of the previous task. After the detection of the feature points, their description is still required.

1. To the function from the previous task, **add a function** that creates descriptors of keypoints. A descriptor is an image patch from the neighbourhood of the point with a specified size.
   - input arguments: image, list of point coordinates (the result of the previous function), neighbourhood size,
   - remove points for which the neighbourhood does not fit within the image,
   - use `filter` with a `lambda` function (X,Y – image size, *size* – neighbourhood/patch size).
   - output: a list of point descriptors together with their coordinates.

```
pts = list(filter(lambda pt: pt[0] >= size and pt[0]< Y-size and    pt[1]
    >= size and pt[1] < X-size, zip(pts[0],pts[1])))
```

2. **Add a function** that matches the feature point descriptors from two images:
   - **Input:** two lists of descriptors (from 1) and a number `N`.
   - **Method:** brute force — for each descriptor from the first list, find the most similar one from the second list.
   - **Similarity measure:** vector distance, the sum of absolute values of their differences, or the dot product.
   - **Result:** a list (coordinates of the point pair + value of the measure); return the `N` **best matches**.

   R Remember to sort correctly before selecting the best matches.
3. **Load the images** *fontanna1.jpg* and *fontanna2.jpg*. Find their keypoints using the Harris method with the function from the previous task. For the detected keypoints, create lists of descriptors using the function from point 1. Set the

neighbourhood size to `15` (you can experiment with other values). Find the best matches using the function from point 2. The parameter `n` can be set to `20`.

4. **Display the results.** Among the data for this exercise there is a file called *pm.py*. You can import it into your own file (*import pm*) and use the function *pm.plot_matches* contained in it. However, it may be necessary to adapt this function to the list returned by the function from point 2 (the provided implementation assumes that each pair is stored in the form $([y1,x1],[y2,x2])$, and the images should be in greyscale). You can also implement your own function for visualising the matches.
5. **Repeat the operations** from the previous points for the images *budynek1.jpg* and *budynek2.jpg*. Notice that in these images the buildings are located at a slightly different angle. Has this affected the quality of the fit?
6. **Repeat the operations** from the previous points for the images *fontanna1.jpg* and *fontanna_pow.jpg*. The images differ significantly in scale. Has Harris dealt with such a difference?
7. **Repeat the operations** from the previous points for the images *eiffel1.jpg* and *eiffel2.jpg*. The images differ in brightness and somewhat in scale. Has Harris dealt with such a difference?
8. **Modify the function** that determines feature point descriptions (from 1) to account for affine changes in brightness:

$$w_{aff} = \frac{w - \bar{w}}{|w - \bar{w}|} \tag{6.9}$$

Whereby the mean $\bar{w}$ can be determined as `np.mean(w)`, and `|w - w̄|` as the standard deviation using `np.std(w)`.
Now try repeating the operations for the images listed above. Has there been any improvement?

6.5 ORB (FAST+BRIEF)

Currently, three popular algorithms (and their variants) are commonly used for keypoint detection and description, i.e. SIFT, SURF and ORB. The SIFT method (*Scale-Invariant Feature Transform*) offers high effectiveness and accuracy, but requires substantial computational effort. The SURF method (*Speeded Up Robust Features*) has lower computational complexity, but in selected cases the quality of the obtained results is noticeably worse than with SIFT. The ORB (*Oriented FAST and Rotated BRIEF*) algorithm is an efficient alternative to SIFT and SURF.

The FAST (*Features from Accelerated Segment Test*) detector is used to detect feature points. It is designed to find corners, similar in spirit to the Moravec and Harris corner detectors. It performs this task by comparing the brightness I_c of a given image point with 16 pixels located on the surrounding circle. This is illustrated in Figure 6.1.

A central point is considered a corner if n consecutive pixels on the circle have a brightness lower than $I_c - t$ or higher than $I_c + t$, where t defines a threshold — a parameter of the algorithm. The best quality results can be obtained by taking $n = 9$. However, the risk of unwanted edge responses inherent in corner detectors may be a problem. Therefore, all detected points should be ranked with respect to the Harris measure (6.1), and then the best N results should be selected (n refers to the FAST test, while N denotes the number of retained points). Unfortunately, the resulting corners are not invariant to

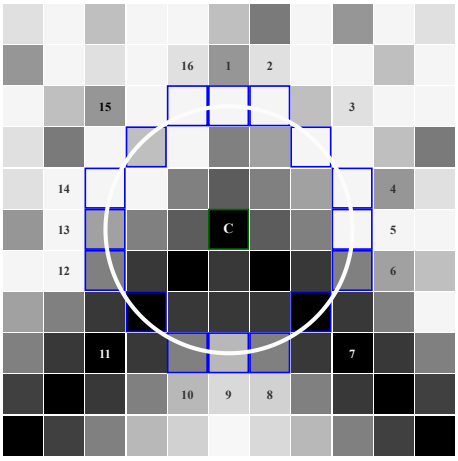

Figure 6.1: Position of the pixels whose brightness is compared with the centre point c in the FAST detector.

image rotation and scale. To address rotation, the *intensity centroid* idea is used. It is based on computing geometric moments of an image patch around a given feature point, defined by the equation:

$$m_{pq} = \sum_{(x,y)\in\Omega} x^p y^q I(x,y) \tag{6.10}$$

where: x, y – local coordinates relative to the detected feature point, $I(x,y)$ – brightness of the pixel at point (x,y), Ω – the set of pixels in the patch.

In this case, the summation proceeds over all pixels within a circle of radius r and centre at a given feature point. From this, the coordinates of the centroid C can be determined:

$$C = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}}\right) \tag{6.11}$$

The orientation of a given feature point is determined by the following:

$$\theta = \text{atan2}(m_{01}, m_{10}) \tag{6.12}$$

In order to make the ORB algorithm robust to scale changes, an image pyramid should be used.

Once a set of feature points has been obtained, the next step is to compute local descriptors. For this purpose, the BRIEF algorithm (*Binary Robust Independent Elementary Features*) is used. It consists in constructing n-element bit strings, according to the relation:

$$f_n(p) := \sum_{i=1}^{n} 2^{i-1} \tau(\mathbf{p}; u_i, v_i) \tag{6.13}$$

where the i-th binary test τ is defined as:

$$\tau(\mathbf{p}; u_i, v_i) := \begin{cases} 1 & \text{if } I(u_i) < I(v_i), \\ 0 & \text{otherwise.} \end{cases}$$

where:
p – an image patch around a given feature point,
u_i, v_i – two pixels inside $\mathbf{p}$ ($u_i \neq v_i$),
$I(w)$ – brightness of the image at a point $w = (x, y)$.

The image patch $\mathbf{p}$ should be blurred to reduce the impact of noise. The blurring is performed using integral images, where a given feature point is represented by the summed brightness of the surrounding pixels (a 5×5 square neighbourhood is assumed for an image patch size of 31×31). Fundamental to the performance of the whole algorithm is also the appropriate selection of pixel pairs for the individual τ tests. The location of the test pairs is defined randomly (Figure 6.2) or deterministically based on a specially developed learning algorithm proposed by the authors of the ORB algorithm[1]. The output of the descriptor is a 256-element binary vector. The Hamming metric is used to determine the similarity between two binary vectors.

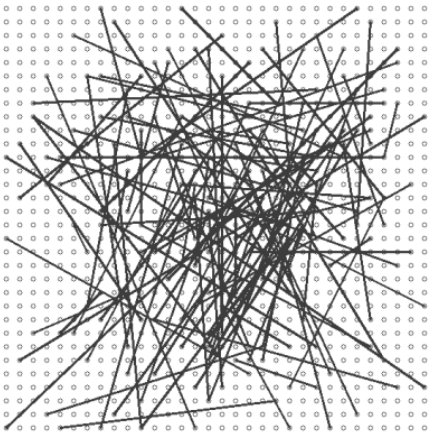

Figure 6.2: Randomly generated point pairs for the BRIEF descriptor.

Exercise 6.3 The aim of the exercise is to learn and implement the elements of the ORB algorithm in a simplified version. The theory described above will be helpful. The results should be compared with the results of the previous exercise.

1. In this task, use the pair of images used in the previous task, e.g. *fontanna1.jpg* and *fontanna2.jpg*.
2. **Implement the FAST detector** to detect feature points and determine the Harris measure for each point. The FAST detector determines a feature point based on an image patch of size 7×7 px.
3. Then, using the *non-maximum suppression* method, perform the first filtering. It consists of removing feature points that are close to each other. Check whether there are multiple detected feature points in a 3×3 px neighbourhood; if so, keep the one with the largest measure.

[1] Rublee, Ethan, et al. "ORB: An efficient alternative to SIFT or SURF." 2011 International conference on computer vision. IEEE, 2011.

4. The next step is to remove feature points that do not have the full 31×31 neighbourhood required by the descriptor.
5. Sort the obtained points according to the Harris measure. Select the N best points.
6. Determine the coordinates of the centroid C and the orientation of the feature point based on the *intensity centroid* idea for each point.
7. **Implement the BRIEF descriptor**. A 31×31 image patch should first be blurred with a 5×5 Gaussian. Point pairs can be determined randomly from an appropriate range or loaded from the file `orb_descriptor_positions.txt`. However, these points are defined for the orientation $0°$. Depending on the angle α obtained from the *intensity centroid*, rotate the point pairs according to the equation:

$$x' = \cos\alpha * x - \sin\alpha * y \tag{6.14}$$
$$y' = \sin\alpha * y + \cos\alpha * x \tag{6.15}$$

   We perform a binary test for all pairs of points, obtaining a 256-bit vector.
8. To compare feature points from two images, use the Hamming metric. A brute force method can be used here — determine the similarity for each pair of points from the two images.
9. Then sort by similarity and select the N best points.
10. **Display the results** and **compare** them with the results from the previous task. Note that the ORB algorithm, compared to the previous task, should handle rotation.

■

6.6 Using feature points to combine images into a panorama

Exercise 6.4 The task will use ready-made functions for feature point detection and descriptors from the OpenCV library.

1. Load images: *left_panorama.jpg* i *right_panorama.jpg*. Convert images to greyscale.
2. Find the feature points in the images. Use a descriptor such as: `cv2.SIFT_create()` or `cv2.xfeatures2d.SURF_create()`.
3. Display the feature points found using the function: `cv2.drawKeypoints`.
4. We then match the feature points from both images. This is what the class is for: cv2.BFMatcher(NORM). For SIFT and SURF the norm will be `cv2.NORM_L2`.
5. There are two matching functions: Brute Force (BF) or KNN. If we choose BF then we call the `.match()` method, if KNN then the `.knnMatch()` method.
6. If BF then we sort the obtained connections by distance (the smaller the distance between the descriptor vectors the better the similarity) and select the N best points. If KNN then we use the following code:

```
best_matches = [ ]
for m,n in matches: # m i~n - best and second best matches
    if m.distance < 0.5*n.distance:  # the best is twice as good as the
        second
        best_matches.append([m])
```

   or the same using list comprehension:

```
best_matches = [ [m] for m,n in matches if m.distance < 0.5*n.distance]
```

7. To check, you can view the received matches – `cv2.drawMatches`.
8. The next step is to determine the rotation and translation of the images (points) with respect to each other, i.e. to determine the homography matrix – function `cv2.findHomography`. Before passing feature points to this function, we need to convert them appropriately. We convert the set of points to a numpy array:

```
keypointsL = np.float32([kp.pt for kp in keypointsL])
keypointsR = np.float32([kp.pt for kp in keypointsR])
```

   and then create sets of corresponding pairs already matched with each other:

```
ptsA = np.float32([keypointsL[m.queryIdx] for m in matches])
ptsB = np.float32([keypointsR[m.trainIdx] for m in matches])
```

9. The final step is to call the function:

```
result = cv2.warpPerspective(image_left, H, (width, height)
```

   where: (width, height) is the sum of the dimensions of the two processed images and the connection to the other image:

```
result[0:image_right.shape[0], 0:image_right.shape[1]] = image_right
```

10. View results.
11. In addition, the actual dimension of the merged images can be detected and the excess black background removed.

■